

NovaPay

2026

Engineering documentation of the
NovaPay cross-border banking
platform: NestJS API with
double-entry ledger, the Expo
mobile app, and the React admin
console.

NovaPay Engineering · September 2026



Table of Contents

1. Executive Overview	2
2. System Architecture	2
2.1 Repository Layout and Infrastructure	2
2.2 Technology Stack	3
2.3 Request Pipeline	3
3. Backend API Reference	4
3.1 Authentication and Sessions	4
3.2 Wallets and Ledger	4
3.3 Payments: Funding and Payouts	5
3.4 Foreign Exchange	5
3.5 Cards (USD)	5
3.6 Receiving: Virtual Accounts	6
3.7 Bills and Airtime	6
3.8 Business Accounts and Bulk Payouts	6
3.9 Stablecoins, Developer API, Fees	7
3.10 Support and Admin Operations	7
4. Database Schema	7
4.1 Core Tables	7
4.2 Design Notes	8
5. Mobile Application	9
6. Admin Console	9
7. Money Movement, FX and Pricing	10
7.1 Double-Entry Ledger	10
7.2 Fee Engine	10
7.3 FX Pricing	10
8. Compliance, Risk and Security	10
8.1 KYC Tiers and Limits	10
8.2 AML, Sanctions and Velocity	11
8.3 Application Security	11
9. Configuration Reference	11
10. Testing and CI	12
11. Deployment and Operations	12
12. Delivery Roadmap and Status	12

1. Executive Overview

NovaPay is a cross-border banking product for users in Nigeria who want to hold, move and grow money in multiple currencies. The platform combines an NGN home rail (funding via Nigerian payment gateways and payouts to Nigerian banks and mobile money) with USD and EUR wallets, virtual foreign accounts for receiving international transfers, virtual (and physical) USD cards, transparent foreign-exchange conversion, stablecoin custody, and business accounts with approval-gated bulk payouts. It is delivered as a monorepo with three applications:

- **apps/api** - a NestJS 11 modular-monolith REST API backed by PostgreSQL (Supabase) and Redis, using Drizzle ORM and a double-entry ledger.
- **apps/mobile** - the customer app, built with Expo (React Native) and expo-router, covering onboarding, wallets, cards, conversions, bills and business features.
- **apps/admin** - an operations console (React 19 + Vite) for KYC review, customer management, transaction inspection, support tickets, AML alerts, disputes and reconciliation.

Development followed a phased roadmap (Phase 0 through Phase 8 for the backend, F0 through F7 for the mobile frontend), each phase documented under `docs/`. All backend phases and all frontend phases are marked complete in their phase documents; partner integrations (Flutterwave, KYC/BaaS/card issuers) are currently wired to mock providers that can be swapped for production providers via environment configuration.

Product	NovaPay cross-border banking (NGN / USD / EUR + USDC / USDT)
Backend	NestJS 11, TypeScript 5.7, Drizzle ORM, PostgreSQL (Supabase), Redis 7
Mobile	Expo 54 / React Native 0.81, expo-router 6, React 19
Admin	React 19 + Vite 6 + React Router 7
Auth	Phone-OTP passwordless, JWT access/refresh with rotation, API keys, step-up
Ledger	Double-entry, minor units (kobo/cents), idempotent transaction creation
Deployment	Docker Compose (Redis + optional Postgres), multi-stage Node 22 image, GitHub Actions CI

Table 1 - Platform summary

2. System Architecture

2.1 Repository Layout and Infrastructure

The repository is an npm monorepo without a root workspace; each application under `apps/` has its own `package.json`, `lockfile` and `scripts`. Infrastructure is defined in `docker-compose.yml`, which runs Redis 7 (port 6379, AOF persistence, healthcheck) and an optional PostgreSQL 16 (port 5432, activated with the `local-db` profile). In practice PostgreSQL is hosted on Supabase: the runtime connection uses the Supabase connection pooler (port 6543) while Drizzle migrations use the direct connection (port 5432).

The API ships as a three-stage `node:22-alpine` Dockerfile (deps, build+prune, runner as non-root node user) exposing port 3000. GitHub Actions CI runs on pushes and PRs to main/develop: it installs dependencies, lints, runs Jest unit tests (with a Redis service container) and builds the API on Node 22.

2.2 Technology Stack

Layer	Technology	Role
apps/api	NestJS 11 / TypeScript 5.7	REST API, modular monolith
Data	Drizzle ORM 0.45 + PostgreSQL (Supabase)	Schema, migrations, queries
Cache/State	Redis 7 (ioredis)	OTP, refresh tokens, idempotency, rate limits, FX cache
Mobile	Expo 54 / RN 0.81 / expo-router 6	Customer app (iOS/Android)
Admin	React 19 + Vite 6 + Router 7	Operations console
Validation	Zod 4 (env) + class-validator (DTOs)	Config and request validation
Observability	nestjs-pino, @nestjs/terminus, Swagger 11	Logging, health checks, API docs
Security	helmet, @nestjs/throttler 6, passport-jwt	Headers, rate limiting, auth
Tests	Jest 30 + supertest	Unit and e2e tests
CI/CD	GitHub Actions	Lint, test, build

Table 2 - Technology stack

2.3 Request Pipeline

Every request flows through the following globally-registered layers, configured in `src/main.ts` and `src/app.module.ts`:

- **helmet** security headers and a 1 MB JSON body limit.
- **ThrottlerGuard** - global rate limiting, with a tighter dedicated limit on OTP endpoints.
- **JwtAuthGuard** - global authentication accepting JWTs (access tokens) or API keys (`np_test_...` / `np_live_...`); routes marked `@Public()` are exempt.
- **IdempotencyInterceptor** - for POST requests carrying an **Idempotency-Key** header, a Redis lock plus a 24-hour cached response makes retries safe.
- **GlobalExceptionHandler** - converts all errors into a uniform envelope `{ success, message, error: { code, message, details } }` using a central error-code catalogue.
- **ValidationPipe** - whitelist-based DTO validation on every route.

Health endpoints (`GET /health` checks both PostgreSQL and Redis; `GET /health/live` is a liveness probe) are provided via Terminus. Swagger documentation is served at `/docs` when `SWAGGER_ENABLED` is set.

3. Backend API Reference

The API is organised into NestJS modules under `apps/api/src/modules/`. All routes are versioned under `/v1`. Amounts are integers in minor units (kobo for NGN, cents for USD/EUR). The tables below list the principal endpoints per module.

3.1 Authentication and Sessions

Authentication is passwordless. A user requests an OTP for their phone number; the code is stored in Redis as a salted SHA-256 hash (pepper + phone + code) with a 300-second TTL and timing-safe comparison. Attempt counters lock out abusive phones. On first successful verification a user record is created (with a unique @tag) and NGN, USD and EUR wallets are provisioned.

JWTs carry sub, phone, kycTier and platformRole claims. Refresh tokens are registered by jti in Redis and rotate on every refresh (the old jti is deleted first); logout revokes the pair and `logout/all` revokes the entire jti set. High-value payouts above `AUTH_STEPUP_PAYOUT_MINOR` require a recent step-up verification.

Endpoint	Description
POST <code>/v1/auth/otp/request</code>	Send a login OTP to a phone number
POST <code>/v1/auth/otp/verify</code>	Verify OTP; creates user + wallets on first login; returns token pair
POST <code>/v1/auth/token/refresh</code>	Rotate refresh token and issue a new access token
POST <code>/v1/auth/logout</code> , <code>/v1/auth/logout/all</code>	Revoke current refresh token / all sessions
POST <code>/v1/auth/step-up/request</code> , <code>/verify</code>	Step-up verification for sensitive operations
GET <code>/v1/auth/me</code> , PATCH <code>/v1/auth/profile</code>	Read and update the authenticated profile
POST <code>/v1/auth/kyc</code>	Submit KYC information

Table 3 - Auth endpoints

SMS delivery for OTPs is pluggable: log, generic HTTP, Twilio and Infobip providers are selected by `SMS_OTP_MODE`; the dev fixed code and the log provider are refused in production and staging environments.

3.2 Wallets and Ledger

The ledger is the system of record. Every money movement creates a transaction row and balanced double-entry `ledger_entries` (debit/credit legs with running `balanceAfter`). System wallets (including a dedicated `fees` wallet) are keyed by `SYSTEM_WALLET_KEYS`. Transaction creation is idempotent via a unique idempotency key. Wallet balances are derived as the sum of ledger entries.

Endpoint	Description
GET <code>/v1/wallets</code>	List the caller's wallets with balances
GET <code>/v1/transactions</code>	Paginated transaction history
GET <code>/v1/transactions/:id</code>	Single transaction detail

Table 4 - Wallet and transaction endpoints

3.3 Payments: Funding and Payouts

Payment rails implement a `PaymentRailProvider` interface with two implementations: a mock rail (for development and tests) and a Flutterwave provider for the NGN rail, selected by `PAYMENT_RAIL=auto|mock|flutterwave`. Funding credits the user's wallet after rail confirmation (webhook or simulation). Payouts run a full risk pipeline: KYC tier limits, sanctions screening, velocity checks, fee computation, step-up verification and ledger holds.

Endpoint	Description
GET /v1/payments/rail	Which payment rail is active
GET /v1/payments/banks/ng	Nigerian bank list for payouts
POST /v1/payments/fund	Initiate a wallet funding (NGN rail)
POST /v1/payments/fund/:id/simulate-complete	Simulate rail completion (mock/dev)
POST /v1/payments/payout	Pay out to a bank account or mobile money
POST /v1/payments/flutterwave/webhook	Signature-verified Flutterwave webhook
GET /v1/payments/flutterwave/return	Redirect landing after hosted funding
POST /v1/beneficiaries, GET /v1/beneficiaries	Manage payout beneficiaries (bank / mobile money)

Table 5 - Payment endpoints

3.4 Foreign Exchange

FX quotes are transparent: the client rate is the mid-market rate (static, HTTP frankfurter, or auto mode, cached in Redis with a configurable TTL) minus a disclosed margin in basis points. A quote is locked for `FX_QUOTE_TTL_SECONDS` and then booked via a double-entry conversion that is balanced per currency through system wallets. Rounding rules are covered by dedicated unit tests (`fx.math.spec.ts`).

Endpoint	Description
GET /v1/fx/rate	Current mid rate for a currency pair
POST /v1/fx/quotes	Request a locked quote with margin applied
POST /v1/fx/quotes/:id/book	Book an open quote; executes the conversion

Table 6 - FX endpoints

3.5 Cards (USD)

Virtual USD cards are issued against a user's USD wallet via a mock card issuer (`mock-card-issuer.ts`). Authorisations place a hold; settlement posts a `card_spend` transaction. PANs are never stored - only brand and last4. Physical cards add shipping fields and a fulfilment status.

Endpoint	Description
POST /v1/cards, GET /v1/cards	Create and list cards
POST /v1/cards/authorizations/:authId/settle	Settle a card authorisation

Endpoint	Description
POST /v1/cards/:id/freeze unfreeze close	Card lifecycle controls
POST /v1/cards/:id/authorize	Simulate a card authorisation (hold)

Table 7 - Card endpoints

3.6 Receiving: Virtual Accounts

Users can be issued USD and EUR virtual accounts through a BaaS provider (mock in this build). Inbound credits - simulated or delivered by the BaaS webhook - land in the matching wallet and pass AML screening. One virtual account per user per currency is enforced.

Endpoint	Description
POST /v1/accounts/virtual, GET /v1/accounts/virtual	Issue and list virtual accounts
POST /v1/accounts/virtual/:id/simulate-inbound	Simulate an inbound international credit
POST /v1/accounts/baas/webhook	Inbound credit webhook from the BaaS provider

Table 8 - Virtual account endpoints

3.7 Bills and Airtime

Endpoint	Description
GET /v1/bills/catalog, GET /v1/airtime/operators	Nigerian bill categories and airtime operators
POST /v1/bills/pay, POST /v1/airtime/topup	Pay a bill / buy airtime (NGN debit)

Table 9 - Bills endpoints

3.8 Business Accounts and Bulk Payouts

Organisations have owner/admin/approver/member roles with per-member `approvalLimitMinor`. Bulk payout batches above an approver's limit must be approved before processing; each item becomes its own transaction with individual failure reasons.

Endpoint	Description
POST/GET /v1/business/organizations	Create and list organisations
POST/GET /v1/business/organizations/:orgId/members	Add and list org members
GET .../wallet, POST .../wallet/fund	Org wallet balance and funding
GET/POST .../bulk-payouts, POST .../bulk-payouts/:batchId/approve	Create, list and approve bulk payout batches

Table 10 - Business endpoints

3.9 Stablecoins, Developer API, Fees

Endpoint	Description
GET /v1/stablecoins, GET /v1/stablecoins/balances	Supported stablecoins and balances
POST /v1/stablecoins/deposit, /v1/stablecoins/convert	Deposit USDC/USDT; convert to/from USD at 1:1 (sandbox custody)
POST/GET /v1/developer/api-keys, POST .../:id/revoke	Programmatic API keys with scopes
POST/GET /v1/developer/webhooks	Register webhook endpoints
GET /v1/public/wallets, GET /v1/public/transactions	API-key-scoped read APIs
GET /v1/fees/quote	Quote the fee for a transaction type and amount

Table 11 - Stablecoin, developer and fee endpoints

3.10 Support and Admin Operations

Endpoint	Description
POST/GET /v1/support/tickets, GET .../:id, POST .../:id/messages reopen	Customer support tickets
GET /v1/admin/kyc, POST /v1/admin/kyc/:id/review	KYC queue and approve/reject (sets tier_1/tier_2)
GET /v1/admin/users, GET/PATCH /v1/admin/users/:id (status, role)	Customer management
GET /v1/admin/transactions, GET .../:id	Platform-wide transactions with ledger legs
GET/POST/PATCH /v1/admin/support/tickets	Support queue and ticket handling
GET /v1/admin/compliance/alerts, PATCH .../:id	AML alert review (dismiss / confirm)
POST /v1/admin/compliance/disputes/:id/resolve	Resolve disputes (won reverses the ledger)
POST /v1/admin/compliance/retention/purge	NDPR data purge of closed users
POST /v1/admin/reconciliation/run, GET .../report/:date	Flutterwave vs ledger reconciliation

Table 12 - Support and admin endpoints

4. Database Schema

The schema (`apps/api/src/database/schema.ts`, migrations in `apps/api/drizzle/`) uses UUID primary keys, bigint minor-unit amounts and timestamptz columns throughout. Twenty-plus tables and enums model the domain; the core financial tables are described below.

4.1 Core Tables

Endpoint	Description
users	phone (unique), email, tag (unique), status, kycTier (tier_0/1/2), platformRole (customer/support/admin)
wallets	ownerType (user/system/organization), userId, systemKey, currency; unique per (owner, currency)
transactions	type (fund, payout, convert, card_spend, receive, bill, airtime, bulk_payout, reversal, ...), status, amount, fee, source/dest wallet, unique idempotencyKey, externalReference
ledger_entries	walletId, transactionId, direction (debit/credit), amount, balanceAfter
beneficiaries	type (bank / mobile_money), bank or MoMo details (mtn/airtel/mpesa/vodafone), currency, country
cards	walletId, processorRef (unique), brand, last4 only, expiry, status, form (virtual/physical), spendLimitMinor, shipping
virtual_accounts	walletId, currency, providerRef, accountNumber/iban/bic; unique (userId, currency)
fx_quotes	pair, amounts, midRate, clientRate, marginBps, status (open/booked/expired), expiresAt, bookedTransactionId
kyc_records	document type/number, verificationStatus, providerRef
organizations / organization_members	org identity + membership with org_role and approval limits
bulk_payout_batches / _items	batch status (draft to completed), items with per-item transaction and failure reason
api_keys / webhook_endpoints	hashed keys with keyPrefix and scopes; webhook URLs with secret and event list
compliance_alerts / card_disputes	AML alerts (rule, severity, status) and card disputes with resolution notes
audit_logs / support_tickets (+_messages)	actor-action-resource audit trail; ticket threads with staff flags

Table 13 - Principal database tables

4.2 Design Notes

- **Money is never stored as float** - all amounts are bigint minor units.
- **Double-entry invariant** - each transaction produces balanced debit/credit legs; conversions are balanced per currency via system wallets.
- **Idempotency at the schema level** - a unique constraint on transactions.idempotencyKey backs the Redis interceptor.
- **Soft lifecycle** - wallets, cards and accounts are closed/frozen, not deleted; user closure feeds the NDPR retention purge.

5. Mobile Application

The customer app is an Expo 54 / React Native 0.81 application with file-based routing (`app/` directory, typed routes, bundle id `com.novapay.app`). State is provided by `AuthContext` and `WalletsContext`; API clients live in `src/lib/` with sessions persisted in Expo SecureStore. The local PIN is salted-SHA-256 hashed and stored in SecureStore, with optional biometric unlock via `expo-local-authentication`.

Endpoint	Description
Onboarding (auth group)	welcome carousel, phone entry (E.164), OTP verification, PIN setup/unlock, KYC document flow
Home (tabs/index)	balance hero, currency strip, quick actions (Add, Convert, Pay, Receive), recent activity
Send / Beneficiaries	send-money, add-beneficiary; payouts gated by KYC tier
Fund / Convert	add-money keypad flow; conversion with locked FX quote and countdown
Cards	cards list, create-card, card-details (freeze/unfreeze/close, simulated purchase)
Receive	USD/EUR virtual account details and simulated inbound credit
Activity / Bills	filterable transaction history with day groups; bills and airtime purchase
Stablecoins	USDC/USDT deposit and convert
Business	business and business-org screens: orgs, members, funding, bulk payouts and approvals
Profile & Help	edit-profile, change-pin, help centre and help-ticket creation

Table 14 - Mobile screens by feature area

6. Admin Console

The operations console (React 19 + Vite + React Router 7) authenticates with the same phone-OTP flow against platform roles; admin phones are seeded via `ADMIN_SEED_PHONES` (default seed: `+2348000000001`). In development, `/v1` requests are proxied to the API on port 3000 and the session token is held in `localStorage`.

Endpoint	Description
LoginPage	Phone + OTP sign-in for staff roles
CustomersPage / CustomerDetailPage	Search customers, view wallets, suspend/close accounts, see open tickets
KycPage	Manual KYC approve/reject, moving users to tier_1 or tier_2
TransactionsPage / TransactionDetailPage	Platform-wide search and filters; per-transaction ledger legs
TicketsPage / TicketDetailPage	Support queue with full message thread and replies

Endpoint	Description
AlertsPage	AML alert review: dismiss, or confirm and act
DisputesPage	Resolve card disputes; a "won" resolution triggers a ledger reversal
ReconPage	Run a reconciliation job and view daily Flutterwave-vs-ledger reports

Table 15 - Admin console pages

7. Money Movement, FX and Pricing

7.1 Double-Entry Ledger

All balances derive from ledger entries, never from mutable balance columns. Funding credits a user wallet and debits a system clearing wallet; payouts do the reverse; conversion legs are balanced per currency through per-currency system wallets. Every entry records the running balanceAfter for auditability.

7.2 Fee Engine

Fees are configured per transaction type as basis points plus a flat component, clamped by min/max, e.g. `FEE_FUND_BPS=50` and `FEE_PAYOUT_BPS=100` plus a 50-kobo flat. Fund fees are deducted from the credit; payout fees are added to the debit. Collected fees post to the dedicated `fees` system wallet, and customers can pre-quote via `GET /v1/fees/quote`.

7.3 FX Pricing

Mid-market rates come from a static table (e.g. `FX_USD_NGN_MID=1600`), an HTTP provider (frankfurter) or auto mode, cached in Redis. The customer rate applies a disclosed margin (`FX_MARGIN_BPS=150` default). Quotes are locked for 60 seconds by default and booked atomically.

8. Compliance, Risk and Security

8.1 KYC Tiers and Limits

Users carry a KYC tier (`tier_0/1/2`) enforced on payouts and wallet balances - for example `tier_0` cannot pay out and its wallet is capped at NGN 50,000, while higher tiers unlock single/daily/monthly payout limits. KYC can be auto-approved in development (`KYC_AUTO_APPROVE`) and is manually reviewed in the admin console otherwise.

8.2 AML, Sanctions and Velocity

Payouts and card authorisations pass sanctions screening (mock or HTTP provider) and velocity checks backed by Redis counters. Large-amount activity raises compliance alerts for admin review. Won disputes trigger automatic ledger reversals. A retention service purges data for closed users in line with NDPR.

8.3 Application Security

- OTPs hashed with a pepper, timing-safe compared, with logout; log-mode SMS disabled in production.
- JWT refresh rotation with server-side jti registry; full session revocation supported.
- API keys stored only as hashes with a visible prefix; scopes attached at resolution time.
- Step-up authentication for payouts above a configurable threshold.
- Global throttling, helmet headers, structured logging with auth-header redaction, and Zod-validated environment config.

9. Configuration Reference

Configuration is loaded from environment variables and validated at boot with Zod (config/env.validation.ts). The template is .env.example. Key groups:

Endpoint	Description
Database	DATABASE_URL (pooler), DATABASE_URL_DIRECT (migrations)
Redis	REDIS_URL
Auth	JWT secrets and TTLs (15m access / 30d refresh), OTP_PEPPER, OTP_MAX_ATTEMPTS, OTP_LOCKOUT_SECONDS, AUTH_STEPUP_PAYOUT_MINOR
OTP delivery	SMS_OTP_MODE=log http twilio infobip, OTP_DEV_FIXED_CODE (dev only)
KYC	KYC_AUTO_APPROVE, KYC_PROVIDER
Payments	PAYMENT_RAIL=auto mock flutterwave, FLUTTERWAVE keys
FX	FX_RATE_MODE, FX_USD_NGN_MID, FX_USD_EUR_MID, FX_MARGIN_BPS, FX_QUOTE_TTL_SECONDS, FX_HTTP_URL
Fees	FEE_FUND_BPS, FEE_PAYOUT_BPS (+flat), per-type schedules
Idempotency	IDEMPOTENCY_TTL_SECONDS (default 86400)
Partners	CARD_ISSUER, BAAS_PROVIDER, MOMO_AGGREGATOR, SANCTIONS_PROVIDER
Ops	THROTTLE settings, SWAGGER_ENABLED, ADMIN_SEED_PHONES

Table 16 - Environment configuration groups

10. Testing and CI

Unit tests (Jest) cover the financially critical logic: OTP hashing and TTL parsing, fee math, FX conversion and rounding, ledger balance math, pagination, the error envelope, environment validation, compliance rules and an auth-to-payment integration spec. End-to-end specs run against a live API: bootstrap/health checks and a full OTP login to funding to payout journey. CI (GitHub Actions) executes lint, unit tests with a Redis service container, and a TypeScript build on Node 22 for every push and PR to main/develop. The mobile and admin apps currently have no automated tests.

11. Deployment and Operations

- **Local stack:** `docker compose up -d` starts Redis; add `--profile local-db` for a local PostgreSQL instead of Supabase.
- **API image:** multi-stage node:22-alpine build; run with `node dist/main.js` as a non-root user on port 3000.
- **Migrations:** Drizzle Kit against the direct database URL (`drizzle-kit migrate/generate/push/studio`).
- **Health:** `/health` (DB + Redis) and `/health/live` for load-balancer probes.
- **API docs:** Swagger UI at `/docs` when enabled.

12. Delivery Roadmap and Status

Endpoint	Description
Phase 0 - Foundations	NestJS scaffold, config validation, Supabase/Drizzle, Redis, health, logging, Docker, CI
Phase 1 - MVP backend	Phone-OTP auth, KYC auto-approve, NGN wallet, double-entry ledger, mock rail fund/payout
Phase 2 - FX + cards	Transparent FX quotes, NGN/USD conversion, virtual USD card issuer
Phase 3 - Receive + bills	USD/EUR virtual accounts, inbound rails, MoMo beneficiaries, bills/airtime, EUR wallet
Phase 4 - Business + API	Organisations, approval-limited bulk payouts, stablecoins, physical cards, API keys + webhooks
Phase 5 - Hardening	Error envelope, pagination, idempotency, throttling, Swagger, tests in CI
Phase 6 - Pricing	Fee engine, fees system wallet, live FX mids with margin disclosure, reconciliation
Phase 7 - Compliance	KYC tier limits, sanctions, AML alerts, velocity controls, disputes, NDPR retention

Endpoint	Description
Phase 8 - Auth hardening	Hashed OTPs, lockout, refresh rotation, pluggable SMS, step-up, hashed mobile PIN
F0-F7 - Mobile frontend	Design system, onboarding, wallets/funding, convert/send, cards, bills/receive, stablecoins, business
Admin console	Customers, KYC review, transactions, support, AML alerts, disputes, reconciliation

Table 17 - Phase status summary

Open items noted in the phase documents: selecting production partners (KYC, BaaS, card issuer, MoMo aggregator, sanctions) and regulatory licensing for Supabase-hosted data; all integrations currently run behind mock providers switchable via configuration.